Taskwarrior Nautical v5.0.0 - CheatSheet

Nautical adds dependable recurrence to Taskwarrior. It keeps Taskwarrior as the task database and uses hooks to create the next task when the current one is completed or stopped.

The core idea is simple:

- Use `cp` when the next task should be based on when you complete this one.
- Use `anchor` when the next task should land on calendar positions.
- Use files when the calendar comes from an external list of dates.
- Use `omit` / `omit_file` to remove dates from an anchor schedule.

Nautical carries the useful task context forward: project, tags, UDAs, dependencies, annotations, chain metadata, and due logic.

---

1. Installation

Nautical consists of:

- `nautical_core/`
- `on-add-nautical.py`
- `on-modify-nautical.py`
- `on-exit-nautical.py`

Install them like this:

```
~/.task/nautical_core/
~/.task/hooks/on-add-nautical.py
~/.task/hooks/on-modify-nautical.py
~/.task/hooks/on-exit-nautical.py
```

Make the hooks executable:

```
chmod +x ~/.task/hooks/on-*-nautical.py
```

Add the required UDAs to `~/.taskrc`:

```
uda.cp.type=string
uda.cp.label=Chain Period

uda.chain.type=string
uda.chain.label=Chain Status
uda.chain.values=on,off
uda.chain.default=off

uda.anchor.type=string
uda.anchor.label=Anchor
uda.anchor_file.type=string
uda.anchor_file.label=Anchor File
```

```
uda.anchor_mode.type=string
uda.anchor_mode.values=flex,all,skip
uda.anchor_mode.default=skip
uda.anchor_mode.label=Anchor Mode

uda.bc.type=string
uda.bc.label=Business Calendar

uda.omit.type=string
uda.omit.label=Omit
uda.omit_file.type=string
uda.omit_file.label=Omit File

uda.chainMax.type=numeric
uda.chainMax.label=Chain Max
uda.chainUntil.type=date
uda.chainUntil.label=Chain Until

uda.prevLink.type=string
uda.prevLink.label=Previous Link
uda.nextLink.type=string
uda.nextLink.label=Next Link
uda.link.type=numeric
uda.link.label=Link Number
uda.chainID.type=string
uda.chainID.label=ChainID
```

Use `cp` as a `string` UDA. Taskwarrior's native `duration` type cannot accept Nautical sequences like `3d,20d,7d` or random periods like `rand(3d..7d)`.

---

2. Configuration

Nautical reads `config-nautical.toml` from the Taskwarrior/Nautical install area. If the file is missing, built-in defaults are used.

Minimal useful config:

```
tz = "Europe/Bucharest"
wrand_salt = "nautical|wrand|v4"

anchor_file_dir = ""
omit_file_dir = ""

enable_anchor_cache = true
anchor_cache_dir = ""

chain_color_per_chain = true
show_timeline_gaps = true
show_analytics = false
analytics_style = "clinical"

panel_mode = "rich"
fast_color = true
```

```
spawn_queue_max_bytes = 524288
max_chain_walk = 500
max_anchor_iterations = 128
max_link_number = 10000

[anchor_presets]
payday = "m:15,-1bd"
workout = "w:mon,wed,fri"

[omit_presets]
holidays = "y:12-24..12-31"
april = "y:apr"

[business_calendar.work]
anchor = "w:mon..fri"
omit = ["y:01-01", "y:12-25"]
# anchor_file = ["extra-open-days.csv"]
# omit_file = ["holidays.csv", "company-closures-*.csv"]
```

Important config notes:

- `tz` is the local timezone used for calendar reasoning and display.
- `wrand_salt` controls deterministic random schedules. Keep it stable if existing random chains should keep their future sequence.
- `anchor_file_dir` and `omit_file_dir` are trusted directories for date files.
- `[business_calendar.<name>]` defines one named calendar for the task-level `bc` UDA.
- `panel_mode` can be `rich`, `fast`, or `line`.
- `show_analytics = false` disables analytics output and computation.
- `NAUTICAL_DIAG=1` prints diagnostic details to stderr.

Named business calendars

A calendar combines open-day and closed-day sources as:

```
(anchor union anchor_file) - (omit union omit_file)
```

The four fields accept either one expression or an array of expressions. `anchor_file` reads from `anchor_file_dir`; `omit_file` reads from `omit_file_dir`. Their file expressions support `*` and `?`, and every pattern must match at least one file.

```
task add "Submit payroll" anchor:"m:-1bd@t=16:00" bc:work
```

An empty `bc` uses the built-in Monday-Friday calendar. Names are case-insensitive and normalized; unknown names are rejected. The selected calendar controls business-day ordinals and all `bd` filters, rolls, offsets, file modifiers, previews, and future chain links.

Calendar definitions must be deterministic. Do not use `/N`, random selectors, `@t=`, business-day ordinals, or business-day modifiers inside their four fields; those forms are rejected because a calendar cannot depend on the business days it is defining.

3. The Recurrence Model

Nautical has two recurrence engines.

`cp`: period chains

`cp` means "create the next link after this much time from completion."

```
task add "Trim the grass" cp:12d due:tomorrow+9h
```

When the task is completed, Nautical creates the next one due 12 days later.

`anchor`: calendar chains

`anchor` means "create the next link on the next matching calendar date."

```
task add "Gym" anchor:"w:mon,wed,fri" due:today+9h
```

Nautical walks the local calendar and finds the next Monday, Wednesday, or Friday.

File-backed dates

`anchor_file` adds explicit dates from a file:

```
task add "Company event prep" anchor_file:"2026.csv@t=12:00,18:00"
```

`omit_file` removes dates from the final schedule:

```
task add "Date night" anchor:"m:rand + w:sat" omit_file:2026.csv
```

The complete anchor formula

For anchor recurrence, Nautical evaluates:

```
(anchor union anchor_file) - (omit union omit_file)
```

That means:

- `anchor` and `anchor_file` both add candidate dates.
- `omit` and `omit_file` remove dates after inclusion is calculated.
- Duplicate local datetimes are deduplicated.
- Omitted dates can still be shown in timelines as skipped/omitted rows.

Use either `cp` or anchor recurrence on a task, not both.

---

4. `cp` Period Chains

Use `cp` for schedules driven by completion.

Fixed period

```
task add "Take vitamin" cp:28h due:today+15h
```

Common periods:

- `12d`
- `2w`
- `28h`
- `90m`
- `P12D`
- `PT28H`

Multiples of 24 hours, such as `1d`, `2d`, and `1w`, keep the seed due time. Odd spans such as `28h` or `33h` are exact additions from completion.

If you want exact-add behavior with a day-like period, use a small offset:

```
task add "Exact cadence" cp:24h+1s due:today+9h
```

Sequential periods

Use a comma-separated sequence when each link has a different interval:

```
task add "Insect treatment" cp:"3d,20d,7d,10d,3d" due:today
```

Nautical uses one period per link and then repeats:

```
#1 -> #2: 3d
#2 -> #3: 20d
#3 -> #4: 7d
#4 -> #5: 10d
#5 -> #6: 3d
#6 -> #7: 3d
```

Panels show the current sequence step:

```
Step 3/5 (7d)
```

Random period

Use `rand(low..high)` for bounded random intervals:

```
task add "Check trap" cp:"rand(3d..7d)" due:today
```

Use jitter when the intent is "roughly every N days":

```
task add "Routine inspection" cp:"14d~2d" due:today
```

`14d~2d` means a deterministic random pick between `12d` and `16d`.

Random `cp` is deterministic per chain link. Nautical can recompute the same link and get the same period again.

---

5. Anchor Grammar

Use anchors for real calendar positions.

Practical grammar:

```
expression = branch ("|" branch)*
branch     = factor ("+" factor)*
factor     = atom | "(" expression ")" ("@" modifier)*
atom       = family ["/N"] ":" spec ["@" modifier]*
family     = "w" | "m" | "y"
```

Read it this way:

- `|` means either/or.
- `+` means must also match.
- `+` binds tighter than `|`.
- Parentheses make grouping explicit.
- A comma belongs inside one atom. It is a list, not top-level OR.

Examples:

```
w:mon,wed,fri
```

Mondays, Wednesdays, or Fridays.

```
w:mon,wed,fri + y:apr
```

Mondays, Wednesdays, or Fridays in April.

```
w:mon | w:wed | w:fri + y:apr
```

Mondays, or Wednesdays, or Fridays in April. This is different because `+` binds before `|`.

```
(w:mon | w:wed | w:fri) + y:apr
```

The explicit grouped form.

Always quote expressions containing `+`, `|`, spaces, or parentheses:

```
task add "Workout" anchor:"w:mon,wed,fri + y:apr"
```

---

6. Weekly Anchors

Weekly anchors use `w:`.

```
w:mon
w:mon,fri
w:mon..fri
w:wk
w:wd
w:we
w/2:mon,thu
w:rand
w:2rand
```

Examples:

```
task add "Gym" anchor:"w:mon,fri" due:today+9h
task add "Study block" anchor:"w:mon..wed"
task add "Field inspection" anchor:"w:2rand@bd@t=09:00"
task add "Split training" anchor:"w:mon@t=09:00,fri@t=15:00"
```

Useful weekly forms:

- `w:mon,fri` means Monday and Friday.
- `w:mon..fri` means Monday through Friday.
- `w:wk` and `w:wd` mean weekdays.
- `w:we` means weekend.
- `w/2:mon` means every 2 weeks on Monday.
- `w:rand` means one random day each ISO week.
- `w:2rand` means two distinct random days each ISO week.

---

7. Monthly Anchors

Monthly anchors use `m:`.

By date

```
m:1
m:15
m:-1
m:ld
m:1,15,-1
m:1..7
m:5bd
m:lbd
m:rand
m:3rand
m/2:-1
```

Examples:

```
task add "Billing sweep" anchor:"m:1,-1" anchor_mode:all due:today
task add "Payroll" anchor:"m:1@nbd@t=09:00" anchor_mode:all due:today
task add "Monthly sampling" anchor:"m:3rand + w:mon..fri"
```

Useful monthly date forms:

- `m:1` means the 1st of the month.
- `m:-1` and `m:ld` mean the last day.
- `m:1..7` means any date from the 1st through the 7th.
- `m:5bd` means 5th business day.
- `m:lbd` means last business day.
- `m:rand` means one random day each month.
- `m:3rand` means three distinct random days each month.

By weekday position

```
m:2sat
m:last-fri
m:1wed,3fri
m/2:2sat
```

Examples:

```
task add "Sourdough bake day" anchor:"m:2sat"
task add "Design session" anchor:"m:last-fri" anchor_mode:all due:today
task add "Parent meeting" anchor:"m:1wed,3fri"
```

---

8. Yearly Anchors

Yearly anchors use `y:` and `MM-DD` date style.

```
y:05-20
y:01-15,04-15,07-15,10-15
y:04-20..05-15
y:10-rand
y:rand
y:2rand
y:02-29
y:q1
y:q1s
y:q1m
y:q1e
```

Examples:

```
task add "Anniversary dinner" anchor:"y:05-20" anchor_mode:all due:today
task add "Quarterly review" anchor:"y:01-15,04-15,07-15,10-15" anchor_mode:all due:today
task add "Seasonal audits" anchor:"y:2rand + y:apr,jul,oct"
task add "Quarter closeout" anchor:"y:q1e..q4e + m:lbd" anchor_mode:all due:today
```

Useful yearly forms:

- `y:05-20` means May 20.
- `y:04-20..05-15` means Apr 20 through May 15.

- `y:10-rand` means one random day in October.
- `y:rand` means one random day in the year.
- `y:2rand` means two distinct random days each year.
- `y:2rand + y:apr,jul,oct` means two random dates selected from April, July, or October.
- `y:02-29` appears only in leap years.

---

9. Modifiers

Modifiers attach with `@`.

```
@t=09:00
@t=09:00,17:30
@nbd
@pbd
@nw
@bd
@+2d
@-2d
@+2bd
@-2bd
@next-mon
@prev-fri
```

Examples:

```
task add "Payroll" anchor:"m:1@nbd@t=09:00"
task add "Reminder" anchor:"y:12-31@prev-fri"
task add "Window" anchor:"m:1@nbd@+1d"
task add "Month-end preparation" anchor:"m:-1@pbd@-2bd"
task add "Shared time" anchor:"(w:mon | w:fri)@t=09:00"
```

Meaning:

- `@t=HH:MM` sets the time for that atom.
- `@nbd` rolls a date to the next business day.
- `@pbd` rolls a date to the previous business day.
- `@nw` rolls to the nearest weekday.
- `@bd` filters to weekdays only.
- `@+Nd` and `@-Nd` shift the final matched date.
- `@+Nbd` and `@-Nbd` shift the final matched date by business days, excluding the matched date itself.
- `@next-mon` and `@prev-fri` roll to a named weekday.

Without `bc`, business days mean Monday through Friday. A task with `bc:<name>` uses that named calendar, including its user-defined closures and exceptional open dates. Date transforms always run in this order: roll, calendar-day offset, then business-day offset. Thus `m:-1@pbd@-2bd` rolls month-end backward if needed and then moves two additional business days earlier.

Do not confuse `m:5bd`, which selects the 5th business day of a month, with `@+5bd`, which shifts an already matched date forward five business days.

Parenthesized OR groups accept shared time, roll, filter, calendar-offset, and business-day-offset modifiers:

```
(w:mon | w:fri)@t=09:00
(y:12-24 | y:12-31)@pbd@-1bd
```

Shared `@t=` also works on groups containing `+`. Date modifiers on `AND` groups must remain on individual atoms because distributing a roll across an intersection could create false matches. Grouped date modifiers also cannot be combined with date modifiers already inside a branch.

---

10. Random Anchors

Random schedules behave like dice rolls, but Nautical makes them reproducible.

The draw is based on:

- the random algorithm version
- `wrand_salt`
- `chainID`
- normalized expression
- recurrence period

This means:

- Two different chains can pick different dates.
- The same chain and period always recompute the same date.
- Preview, completion, timeline, retries, and sync agree without storing mutable random state.
- Changing `wrand_salt` intentionally changes future random sequences.

Examples:

```
w:rand
w:2rand
m:rand
m:3rand
m:rand + w:sat,sun
m:rand + (w:sat | w:sun)
y:rand
y:2rand
y:2rand + y:apr,jul,oct
```

Important distinction:

```
m:rand + w:sat,sun
```

One random weekend date per month.

```
m:rand + (w:sat | w:sun)
```

One random Saturday branch and one random Sunday branch, so it can produce two dates per month.

Counted random selects without replacement. If a period has fewer eligible dates than requested, that period is skipped.

---

11. Anchor Files and Omit Files

Use files when the calendar already exists somewhere else: holidays, company dates, school terms, farming schedules, maintenance windows.

Configure directories:

```
anchor_file_dir = "/home/user/.task/nautical_anchors"
omit_file_dir = "/home/user/.task/nautical_omits"
```

Accepted file formats:

```
2026-01-01
2026-04-20..2026-05-15
# comments are ignored
```

CSV is also accepted when it has a `date` column:

```
date,description
2026-01-01,New Year
2026-12-25,Christmas
```

Column order does not matter. Extra columns are ignored.

Examples:

```
task add "Event prep" anchor_file:"2026.csv@-1d@t=12:00,18:00"
task add "Early event prep" anchor_file:"2026.csv@-2bd@t=12:00"
task add "Workout" anchor:"w:mon,wed,fri" omit_file:"holidays.csv"
task add "Date night" anchor:"m:rand + w:sat" omit_file:"2026.csv"
```

File modifiers:

- `@-1d` shifts dates back one day.
- `@+2d` shifts dates forward two days.
- `@-2bd` shifts dates back two business days.
- `@+2bd` shifts dates forward two business days.
- `@t=12:00,18:00` creates one occurrence per time on each file date.

`anchor_file` adds dates. `omit_file` removes dates.

---

12. Omit Rules

`omit` removes dates from an anchor recurrence.

```
task add "Workout" anchor:"w:mon,wed,fri" omit:"w:wed"
```

This keeps Monday and Friday, removes Wednesday.

More examples:

```
task add "Workout" anchor:"w:mon,wed,fri" omit:"w:wed + y:apr,jul,oct"
task add "Billing" anchor:"m:1" omit:"y:01-01"
task add "Date night" anchor:"m:rand + w:sat" omit:"y:12-24..12-31"
```

Rules:

- `omit` uses the same date grammar as `anchor`.
- `omit` applies after `anchor` and `anchor_file` are merged.
- `omit` removes dates, not individual times.
- Timed omit expressions such as `omit:"w:mon@t=09:00"` are rejected.
- Omitted dates are skipped; Nautical keeps searching for the next valid date.

---

13. Anchor Modes

Anchor mode controls what happens when a task is completed late.

```
anchor_mode:skip
anchor_mode:all
anchor_mode:flex
```

Meaning:

- `skip` jumps to the next future match. This is the default and works well for routines.
- `all` backfills every missed anchor. This is useful for bills or compliance work.
- `flex` skips backlog once, then returns to `all`.

Examples:

```
task add "Gym" anchor:"w:mon,wed,fri" anchor_mode:skip
task add "Monthly billing" anchor:"m:1" anchor_mode:all
task 42 modify anchor_mode:flex
```

---

14. Limits and Manual Stops

Limits work with both `cp` and anchor recurrence.

```
chainMax:N
chainUntil:YYYY-MM-DD
chainUntil:YYYY-MM-DDTHH:MM
```

Examples:

```
task add "Bootcamp" anchor:"w:mon,fri" chainMax:6 due:today
task add "Deep work" cp:1d chainUntil:2026-12-31T12:00 due:today+12h
```

Rules:

- `chainMax` stops after the N-th link.

- `chainUntil` includes matches up to that local date/time.
- If both are set, Nautical stops at whichever comes first.
- If the current link is overdue, Nautical does not spawn a new pending link until you complete or delete it.

Manual stops:

```
task 42 modify chain:off
task 42 delete
```

Nautical shows a panel when recurrence is disabled or when a pending chain task is deleted.

---

15. Panels and Timelines

Nautical panels are meant to answer:

- What pattern is this task using?
- What is the next link?
- What was skipped or omitted?
- How far apart are the occurrences?
- Is the chain near a limit?

For anchor chains, timelines can show:

- completed rows
- the active next row
- future rows
- skipped rows
- omitted rows

Config controls:

```
panel_mode = "rich"      # rich, fast, or line
show_timeline_gaps = true
show_analytics = true
chain_color_per_chain = true
```

Use `show_analytics = false` if you want less output and less computation on completion.

---

16. Sync and Safety

Nautical is designed for synced Taskwarrior setups.

Key safety rules:

- Each chain has a `chainID`.
- Each task has a numeric `link`.
- Parent/child links use `prevLink` and `nextLink`.
- Nautical checks for existing children before spawning.
- Completion spawns are queued and drained safely.

- If a spawn cannot be completed, it is retained for retry or written to dead-letter state.

Important sync behavior:

Nautical carries forward the task data available on the device that completes and syncs the task. If a task is annotated on one device, completed on another, and then edited again elsewhere, the next link reflects the data available to the device that performed the Nautical spawn.

---

17. Operator Tools

Run doctor first when something looks wrong:

```
nautical doctor
```

Use JSON when scripting:

```
nautical doctor --json
```

Doctor is read-only. It checks:

- Taskwarrior availability
- hook installation
- UDA definitions
- config parsing
- queue/dead-letter state
- chain metadata
- safe chain repair opportunities
- hookless completion reconcile plans

Repair chain metadata:

```
nautical chain-repair
nautical chain-repair --apply
```

Reconcile hookless completions:

```
nautical reconcile
nautical reconcile --apply
```

Use the dry-run output first. Apply only after reviewing the proposed changes.

---

18. Troubleshooting

Nautical does not run

Check:

- hooks are executable
- UDAs are installed

- `nautical_core/` is in the expected location
- `NAUTICAL_CORE_PATH` is not pointing to a stale copy

Run:

```
NAUTICAL_DIAG=1 task add "test" anchor:"w:mon"
```

A task does not spawn the next link

Check:

- the task has `chain:on`
- the task has `chainID`
- the current link is not overdue in a way that pauses spawning
- `chainMax` or `chainUntil` did not stop the chain
- `anchor_mode` is what you expect

Run doctor before manually changing chain metadata.

Taskwarrior native recurrence behaves differently

Nautical is separate from Taskwarrior's built-in recurrence. Nautical tasks use Nautical UDAs such as `cp`, `anchor`, `anchor_file`, `omit`, `bc`, `chainID`, and `link`.

Random dates look surprising

Random does not mean balanced. Repeats and streaks can happen. Nautical guarantees reproducibility and valid candidate selection, not visual evenness.

A date file is rejected

Check:

- the file is inside `anchor_file_dir` or `omit_file_dir`
- the task uses only a basename, not a path
- CSV has a `date` column
- dates are valid `YYYY-MM-DD`

---

19. Quick Recipe Library

`cp`

```
task add "Trim the grass" cp:12d due:tomorrow+9h
task add "Take vitamin" cp:28h due:today+15h
task add "Insect treatment" cp:"3d,20d,7d,10d,3d" due:today
task add "Check trap" cp:"rand(3d..7d)" due:today
task add "Routine inspection" cp:"14d~2d" due:today
task add "Client follow-up" cp:"1d,3d,7d,14d,30d" chainMax:5 due:today
```

Weekly

```
task add "Gym" anchor:"w:mon,fri" due:today+9h
task add "Study" anchor:"w:mon..wed"
task add "Split training" anchor:"w:mon@t=09:00,fri@t=15:00"
task add "Field inspection" anchor:"w:2rand@bd@t=09:00"
```

Monthly

```
task add "Billing sweep" anchor:"m:1,-1" anchor_mode:all due:today
task add "Payroll" anchor:"m:1@nbd@t=09:00" anchor_mode:all due:today
task add "Month-end preparation" anchor:"m:-1@pbd@-2bd" due:today
task add "Design session" anchor:"m:last-fri"
task add "Monthly sampling" anchor:"m:3rand + w:mon..fri"
```

Yearly

```
task add "Anniversary" anchor:"y:05-20" anchor_mode:all due:today
task add "Quarterly review" anchor:"y:01-15,04-15,07-15,10-15"
task add "Seasonal audits" anchor:"y:2rand + y:apr,jul,oct"
task add "Quarter closeout" anchor:"y:q1e..q4e + m:lbd"
```

Omit and files

```
task add "Workout" anchor:"w:mon,wed,fri" omit:"w:wed"
task add "Workout" anchor:"w:mon,wed,fri" omit:"w:wed + y:apr,jul,oct"
task add "Date night" anchor:"m:rand + w:sat" omit_file:2026.csv
task add "Event prep" anchor_file:"2026.csv@-1d@t=12:00,18:00"
```

---

20. Practical Advice

- Start with simple schedules.
- Quote complex anchors.
- Prefer `omit` over clever negative logic.
- Use files for external calendars.
- Keep `wrand_salt` stable.
- Run doctor before repairing data.
- Use `chainMax` or `chainUntil` for experiments.
- If an expression is hard to explain, split it into two tasks.

---

21. Support

If Nautical improves your workflow and you want to support continued development:

Buy me a book, PayPal, or GitHub Sponsors.

Support helps sustain the time invested in building, testing, and maintaining Nautical.